

Paraphrase Detection and Sonnet Generation with LoRA and DPO

Stanford CS224N {Default}

Jaagat Prashar

Department of Computer Science
Stanford University
jaagat@stanford.edu

Kyle Tianshi

Department of Computer Science
Stanford University
ktianshi@stanford.edu

Diya Bhattacharjee

Department of Computer Science
Stanford University
diyab@stanford.edu

Abstract

In this project, we explore whether combining parameter-efficient fine-tuning (LoRA) and preference-based optimization through Direct Preference Optimization (DPO) while varying hyperparameters improves output quality and generalization in GPT-2 style transformers. We evaluate these methods on three tasks, with paraphrase detection as the primary focus, and sonnet generation and sentiment analysis as secondary. Through these three tasks, we aim to test different functions of the model: sonnet generation requires creative language modeling and structural control, paraphrase detection evaluates semantic understanding and classification performance, and sentiment analysis assesses the model's capacity to distinguish emotional polarity in text. Our approach employs a GPT-2 baseline trained from scratch and then compared to our extensions. We find that LoRA fine-tuning significantly improves paraphrase detection accuracy over the baseline. DPO with hard negative mining approached but did not exceed LoRA performance, while synthetic corruption-based DPO underperformed, suggesting that corrupted negatives introduce shortcuts that hurt generalization.

1 Key Information to include

- Mentor: Ahmed Ahmed
- External Collaborators (if you have any): N/A
- Sharing project: N/A
- Late Days (if you are using any):

Team member	Late days remaining
Diya Bhattacharjee	2
Kyle Tianshi	6
Jaagat Prashar	0
Total pooled	8
Team size	3
Late days applied for the report	2

2 Introduction

Paraphrase detection software is critical to maintaining the integrity of academic work and enabling search engine optimization, to name a few applications. This project aims to fine-tune a GPT-2 style model to exhibit robust performance on paraphrase detection. We train the model from scratch, finetune it on the Quora dataset, and implement Low-Rank Adaptation (LoRA) and Direct Preference Optimization (DPO) to improve training efficiency and efficacy. While other paraphrase detection models rely on bidirectional encoders such as BERT, our model frames paraphrase detection as a generative cloze-style task using a decoder-only GPT-2 architecture, and explores multiple strategies for improving performance, including parameter-efficient fine-tuning via LoRA and preference-based optimization via DPO with two distinct triplet construction methods.

Our project also tests the GPT-2-style model’s ability to perform short-form creative generation and sentiment analysis. We finetune the model on a dataset of Shakespeare’s sonnets and measure how closely the generated poems follow Shakespeare’s language distribution. Separately, we finetune the model on movie reviews and analyze its classification accuracy. Our results demonstrate that carefully designed optimization strategies such as LoRA and DPO paired with varying the hyperparameters and model size can improve model performance.

3 Related Work

Our project employs LoRA and DPO as two main frameworks for further improving training. LoRA is a PEFT method that injects low-rank trainable matrices into the model. LoRA, introduced by Microsoft in 2021, trains millions of parameters instead of billions, enables faster fine-tuning, and easy multi-task adaptation through LoRA adapters.

DPO is a technique for aligning language models using human preference data. This approach skips the steps from RLHF and trains the model to prefer better responses based on a policy that increases the likelihood of preferred responses and decreases the likelihood of unpreferable responses. DPO enables simpler training than RLHF, more stable training, and less infrastructure usage.

We also employ SMART regularization [1], which combines smoothness-inducing adversarial regularization with Bregman proximal point optimization to improve robustness during fine-tuning.

4 Approach

We implemented a pre-trained model with a similar architecture to GPT-2, which served as our baseline [2]. We fine-tuned the model to perform paraphrase detection, sonnet generation, and sentiment analysis. For paraphrase detection and sonnet generation, we implemented Low-Rank Adaptation [3] to enhance the fine-tuned model’s performance by reducing the number of trainable parameters. As shown by the equation below, LoRA doesn’t change the frozen weights, but learns a small set of additional parameters θ . The equation shows how LoRA learns a small low-rank weight update that is added to the frozen base model, so the combined model maximizes the log-likelihood of the correct training tokens.

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log (p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t | x, y_{<t})) \quad (1)$$

For paraphrase detection specifically, we implemented DPO [4], a technique for aligning language models with preference data that avoids the complexity of reinforcement learning from human feedback (RLHF). Rather than requiring a separate reward model, DPO directly optimizes the policy using paired preference examples. In our case, paraphrase pairs serve as chosen responses and non-paraphrase pairs serve as rejected responses. We explored two methods for constructing these preference pairs: hard negative mining, which selects natural non-paraphrase partners with high lexical overlap as rejected examples, and a synthetic corruption approach, which generates rejected examples by randomly shuffling words in the chosen sentence when no suitable natural negative exists. The equation below shows how the gradient of the loss function increases the likelihood of

preferred completions and decreases the likelihood of dispreferred completions.

$$\begin{aligned} \nabla_{\theta} \mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = \\ -\beta \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\underbrace{\sigma(\hat{r}_{\theta}(x, y_l) - \hat{r}_{\theta}(x, y_w))}_{\text{higher weight when reward estimate is wrong}} \left[\underbrace{\nabla_{\theta} \log \pi(y_w | x)}_{\text{increase likelihood of } y_w} - \underbrace{\nabla_{\theta} \log \pi(y_l | x)}_{\text{decrease likelihood of } y_l} \right] \right] \end{aligned}$$

We additionally implemented SMART regularization [1], which combines smoothness-inducing adversarial regularization with Bregman proximal point optimization. Smoothness-inducing regularization perturbs input embeddings and penalizes sensitivity to those perturbations, while Bregman proximal point optimization prevents the model from drifting too far from the previous checkpoint at each update step.

The base GPT-2 architecture and starter code were not written by us; our original contributions are the classifier implementation (attention, layer, etc.), fine-tuning pipeline, and LoRA, DPO, and SMART integration.

5 Experiments

5.1 Data

We used the Sonnet Dataset and the Quora datasets provided by the default project specifications for the sonnet generation and paraphrase detection tasks. We also used the SST and CFIMDB datasets for sentiment analysis.

To construct DPO preference triplets from the Quora training data, we implemented two independent methods. Both methods first build positive and negative partner maps from the labeled pairs. For each anchor sentence, its paraphrase partners (label=1) and non-paraphrase partners (label=0) are indexed separately.

Method 1 (hard negatives only) selects the rejected example for each anchor by finding the non-paraphrase partner with the highest word overlap with the anchor, using a minimum overlap threshold of 0.2 to filter out trivially easy negatives. Anchors with no suitable natural negative are skipped entirely. This yielded $\sim 30\text{k}$ triplets.

Method 2 (hard negatives with synthetic corruptions) follows the same hard-negative mining procedure, but instead of skipping anchors that lack a suitable natural negative, it generates a synthetic rejected example by randomly shuffling 40% of the words in the chosen sentence, preserving vocabulary but destroying syntax. This yielded $\sim 104\text{k}$ triplets.

Each triplet is formatted using the same cloze-style prompt template as the base paraphrase detection task: Question 1: “{anchor}” / Question 2: “{chosen or rejected}” / Are these questions asking the same thing?

5.2 Evaluation method

Our evaluation metric for the paraphrase detection task is comparing the model’s binary output (Yes/No) to the labeled data, measuring the percent accuracy.

Our evaluation metric for the sentiment analysis task is classification accuracy and F1 scores calculated on the SST and CFIMDB datasets. We also evaluate the loss and accuracy at each epoch when training. The F1 score is the harmonic mean of precision and recall:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2)$$

where $\text{Precision} = \frac{TP}{TP+FP}$ and $\text{Recall} = \frac{TP}{TP+FN}$.

Our evaluation metric for sonnet generation is the CHRF score, which measures how closely our sonnets follow Shakespeare’s language distribution using character n-gram F-score.

5.3 Experimental details

For paraphrase detection, we fine-tuned the GPT-2 model (12 layers, 768 hidden dimensions, 12 attention heads) on the Quora dataset using a learning rate of $1e-5$, batch size of 64, and 10 epochs, with training taking approximately 1.5 hours on an H100 GPU, and recorded its performance. Then, we implemented PEFT and recorded the model’s performance (experimenting with a higher learning rate).

For DPO training, we ran both triplet construction methods independently with a learning rate of $1e-5$, batch size of 16, and 10 epochs on an H100 GPU.

For SMART, we trained from scratch on the Quora dataset using a learning rate of $1e-5$, batch size of 16, perturbation radius epsilon of $1e-6$, smoothness weight lambda of 0.5, and Bregman penalty weight mu of 1.0.

For sonnet generation, we finetuned the GPT-2-medium model (24 layers, 1024 hidden dimensions, 16 attention heads) on the Sonnets dataset using LoRA, a learning rate of $3e-5$, batch size of 16, and 10 epochs, with training taking approximately 15 minutes on a T4 GPU. We arrived at this result after experimenting with different hyper-parameters and model sizes. We found that the GPT-2-medium model outperformed the GPT-2-small model across learning rates.

For sentiment analysis, we used a learning rate of $1e-5$ for the full model, a batch size of 64, and trained for 10 epochs on an A100 GPU, with each run taking about 10 minutes. We trained both the last-linear-layer and full-model configurations on SST and CFIMDB.

5.4 Results

Paraphrase Detection

For the paraphrase detection task, results are summarized in Table 1. LoRA fine-tuning improved dev accuracy from 0.82 to 0.87, and scaling to GPT-2-medium further improved dev accuracy to 0.891 with an F1 score of 0.884. DPO with hard negative mining (Method 1) achieved 0.80 dev accuracy, while DPO with synthetic corruptions (Method 2) achieved 0.60 dev accuracy. We additionally ran SMART regularization for one epoch, achieving 0.37 dev accuracy.

Model	Accuracy
Paraphrase Detector Baseline (Dev Set)	0.82
Paraphrase Detector with PEFT (Dev Set)	0.87
Paraphrase Detector with PEFT and GPT-2-medium (Dev Set)	0.891
Paraphrase Detector with DPO Method 1, Hard Negatives (Dev Set)	0.80
Paraphrase Detector with DPO Method 2, Synthetic Corruptions (Dev Set)	0.60
Paraphrase Detector with SMART (Dev Set, Epoch 1)	0.37

Table 1: Paraphrase Detection Results

As shown in Figure 1, LoRA with a higher learning rate converges significantly faster than the baseline and achieves a higher final dev accuracy of 0.891 compared to 0.825, demonstrating the effectiveness of parameter-efficient fine-tuning for this task. We experimented with various LoRA configurations including rank, dropout, bias, and learning rate, and found that increasing the rank and alpha while lowering the learning rate produced the best performance.

Sonnet Generation

For the sonnet generation task, we expected that increasing the model size and implementing LoRA to ensure training efficiency would improve performance. It was interesting that the model generated more accurate sonnets on the Dev Set than the Test Set despite the two datasets being roughly equal in size.

Sentiment Analysis

For the sentiment analysis task, results are summarized in Table 3. Full-model fine-tuning outperformed last-linear-layer fine-tuning on both datasets, achieving 0.497 dev accuracy on SST and 0.980 on CFIMDB.

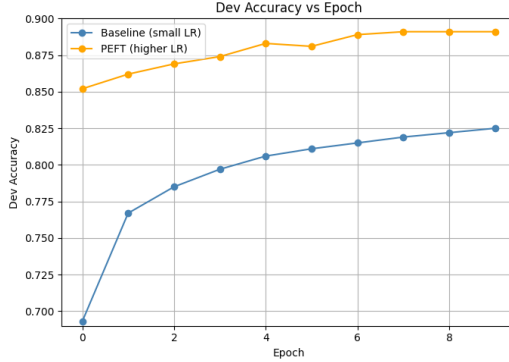


Figure 1: Dev accuracy vs epoch for the baseline (small learning rate) and LoRA with PEFT (higher learning rate). LoRA converges faster and achieves higher final accuracy (0.891) compared to the baseline (0.825).

Model	CHRF Score
Sonnet Generation Baseline (Dev Set)	38.930
Sonnet Generation with Adjusted LR, Model Size, and LoRA (Dev Set)	42.869
Sonnet Generation with Adjusted LR, Model Size, and LoRA (Test Set)	41.663

Table 2: Sonnet Generation Results

6 Analysis

Paraphrase Detection

LoRA fine-tuning improved paraphrase detection accuracy from 0.82 to 0.87 on the dev set, confirming that parameter-efficient fine-tuning is effective for this task. Injecting LoRA into 36 layers of GPT-2-medium further improved dev accuracy to 0.891 with an F1 score of 0.884, suggesting that both model capacity and the number of adapted layers contribute meaningfully to performance. The training curve in Figure 1 reveals that LoRA with a higher learning rate converges much faster than the baseline, reaching 0.852 at epoch 0 compared to 0.693 for the baseline, and continuing to improve steadily through epoch 7. The baseline, by contrast, converges slowly and plateaus around 0.825, never closing the gap with LoRA.

For DPO, neither method reached the baseline accuracy of 0.82. Method 1 (hard negatives only) achieved 0.80 dev accuracy, and Method 2 (hard negatives with synthetic corruptions) achieved 0.60 dev accuracy. The gap between the two methods reveals an important failure mode: when the rejected example is a syntactically corrupted version of the chosen sentence, the model can learn to distinguish chosen from rejected based on grammatical well-formedness rather than semantic content. This is a surface-level shortcut that does not generalize to real paraphrase detection, where both sentences are grammatically valid. Method 1 avoids this shortcut by using real non-paraphrase partners as rejected examples, producing more meaningful preference signal.

The underperformance of both methods relative to even the baseline is likely attributable to the training setup. Since DPO is run independently rather than on top of a LoRA fine-tuned model, it does not benefit from the parameter efficiency and regularization that LoRA provides. Additionally, at a learning rate of $1e-5$, the optimization may be too aggressive for preference-based training, which typically requires a more conservative update schedule than standard fine-tuning. A lower learning rate of $1e-6$ would be a natural next step but was not explored due to time constraints.

It is also worth noting that Method 1 yields only $\sim 30k$ triplets compared to the 141k examples in the Quora training set, meaning the DPO training stage sees significantly less data than the original fine-tuning stage. Increasing triplet coverage, for example by relaxing the minimum overlap threshold of 0.2 or incorporating an external labeled dataset such as MRPC, could provide a richer preference signal and potentially close the gap with the baseline.

Model	SST Dev Acc	CFIMDB Dev Acc
Last-Linear-Layer	0.454	0.873
Full-Model	0.497	0.980
Baseline (Last-Linear-Layer)	0.462	0.861
Baseline (Full-Model)	0.513	0.976

Table 3: Sentiment analysis dev accuracy on SST and CFIMDB.

We also experimented with SMART regularization, which achieved 0.37 dev accuracy after one epoch; we stopped early given the low accuracy relative to the baseline. The SMART paper typically applies regularization on top of an already fine-tuned model rather than from scratch, which likely explains the slow convergence. Applying SMART on top of an existing LoRA checkpoint would be a natural next step.

Sonnet Generation

For sonnet generation, we experimented with different hyperparameters to maximize the model’s learning of the training data while preventing overfitting. Since Shakespeare only wrote 145 sonnets, our training dataset consisted of approximately 130 examples. We chose to use a GPT-2-medium model because we found that it outperformed the GPT-2-small model at the same learning rate with minimal additional compute cost.

We experimented with learning rates ranging from $1e-6$ to $4e-5$, with $1e-5$ serving as our baseline. We found that learning rates below $1e-5$ performed worse than the baseline since they did not sufficiently reduce training loss in 10 epochs. Conversely, learning rates above $3e-5$ were effective at minimizing training loss, but they overfit to the dataset and underperformed compared to the baseline. We thus settled on $3e-5$ as our optimal learning rate.

Sentiment Analysis

Full-model fine-tuning outperformed last-linear-layer fine-tuning on both datasets, consistent with the expectation that updating all parameters allows the model to better adapt its representations to the target task. On CFIMDB, the full model achieved 0.980 dev accuracy, likely because CFIMDB is a binary classification task, making it inherently simpler than SST’s five-category classification. The training logs show the model peaked at epoch 6 before slightly declining, suggesting mild overfitting in later epochs. SST proved more challenging across all configurations, with the best result of 0.497, reflecting the greater difficulty of fine-grained sentiment classification across five categories. Notably, our last-linear-layer implementation underperformed the baseline on both datasets, suggesting that freezing the majority of model parameters is insufficient for this task without further tuning.

7 Conclusion

Our project found that parameter-efficient fine-tuning via LoRA significantly improves paraphrase detection over the baseline GPT-2 model, with further gains from scaling to GPT-2-medium. DPO with hard negative mining approached but did not reach the baseline, while synthetic corruption-based DPO underperformed significantly. This suggests that the quality of negative examples in DPO is critical to the success of preference optimization. For sonnet generation, increasing model size to GPT-2-medium and carefully tuning the learning rate meaningfully improved CHRF score over the baseline. For sentiment analysis, full-model fine-tuning consistently outperformed last-linear-layer fine-tuning, with near-perfect accuracy on the binary CFIMDB task.

As avenues for future work, we would consider training on datasets that enhance robustness in different contexts. One example is the PAWS dataset [5] (Paraphrase Adversaries from Word Scrambling), which challenges the model with adversarially constructed sentence pairs that have high lexical overlap but different meanings, exactly the kind of near-miss the model currently struggles to distinguish. Another example is training the model on a dataset of long forms of text, so it can identify paraphrased content dispersed over paragraphs or pages.

One limitation of the finetuned model for sonnet generation was the lack of original training examples to draw on. The OuPoCo database [6] is a collection of approximately 5,000 French sonnets from the

19th century, which could add examples to our training dataset while testing the model’s multilingual capabilities.

We could consider training the model on modern-day replications of Shakespeare’s sonnets, but this raises an ethical question about whether it is fair to train a model on the original work of living artists who could profit from their poems.

Team contributions (Required for multi-person team)

Diya: Equal contributions to the GPT-2 base model. Finetuned the model on paraphrase detection to obtain the baseline score. Finetuned the model on sonnet generation and implemented extensions to improve on the baseline. Contributed significantly to the Final Report.

Kyle: Equal contributions to the GPT-2 base model. Ran sentiment analysis experiments on SST and CFIMDB. Researched and evaluated candidate datasets for DPO training, including MRPC, ParaSCI, and PAWS. Implemented DPO construction methods for the Quora dataset and SMART for paraphrase detection. Contributed significantly to the Final Report.

Jaagat: Equal contributions to the GPT-2 base model. Fine-tuned GPT-2 using LoRA for paraphrase detection (full fine-tuning and with LoRA). Conducted extensive research for datasets and motivation to use LoRA, and contributed significantly to proposal, milestone, and report.

References

- [1] Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Tuo Zhao. SMART: Robust and efficient fine-tuning for pre-trained natural language models through principled regularized optimization. *arXiv preprint arXiv:1911.03437*, 2019.
- [2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.
- [3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [4] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [5] Yuan Zhang, Jason Baldridge, and Luheng He. Paws: Paraphrase adversaries from word scrambling. *arXiv preprint arXiv:1904.01130*, 2019.
- [6] Iona Lacroix and Alexandre Gefen. Oupoco, the oupoco database. 2022.